

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 6

Roll no: 62 PRN No: 12413457 Name of Student: Swaraj Shedge

Subject Name : Deep Learning

Problem Statement:

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm

1. **Environment & Dependency Initialization:** Import deep learning and data processing libraries (tensorflow, keras, numpy, matplotlib, pathlib).
2. **Parameter Definition:** Set standard image dimensions (224×224 pixels) and batch size (32).
3. **Dataset Ingestion & Stratification:**
 - Load images directly from the dataset directory using `tf.keras.utils.image_dataset_from_directory`.
 - Apply an **80/20** train-validation split with a fixed random seed (seed=42) to ensure reproducibility.
4. **Class Label Extraction:** Identify and index the unique target class labels from subdirectories.
5. **Data Pipeline Construction:** Yield batched tensor datasets (train_dataset and validation_dataset) ready for downstream CNN model ingestion.

CODE:

```
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential

import matplotlib.pyplot as plt

import numpy as np

from pathlib import Path
```

✓ 1m 32.0s

```
dataset_path = Path("tomato")

image_height = 224
image_width = 224
batch_size = 32

train_dataset = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=(image_height, image_width),
    batch_size=batch_size
)

validation_dataset = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=(image_height, image_width),
    batch_size=batch_size
)
```

[2] ✓ 1.2s

```
... Found 18160 files belonging to 10 classes.
Using 14528 files for training.
Found 18160 files belonging to 10 classes.
Using 3632 files for validation.
```

```
class_names = train_dataset.class_names

print("Classes in the Dataset:")
print(class_names)

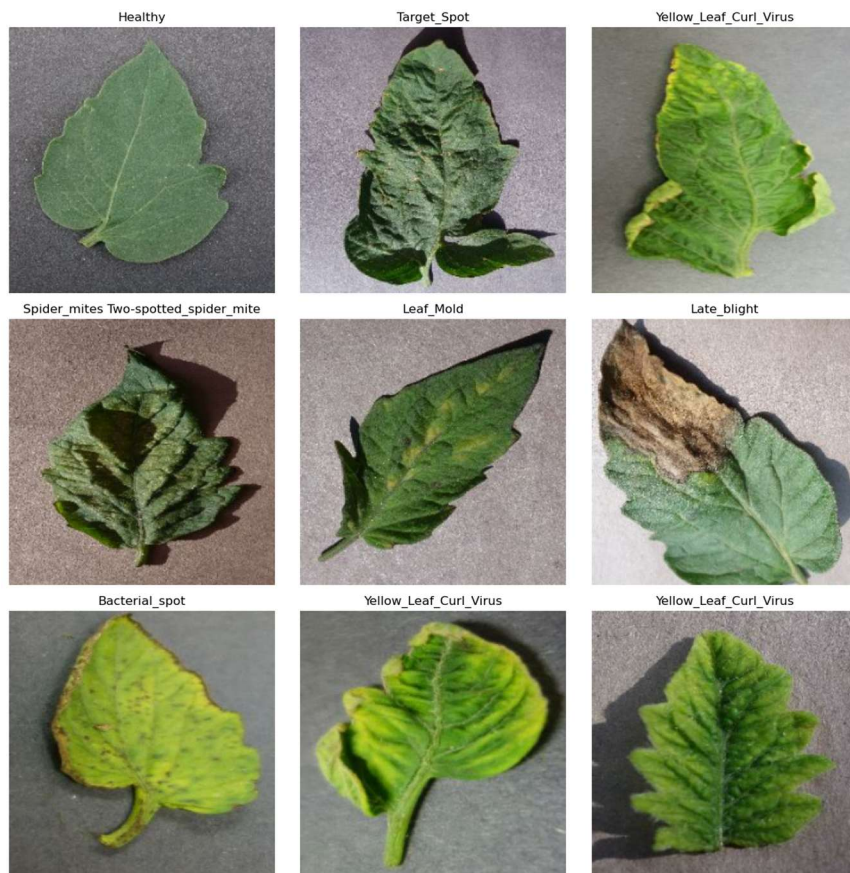
plt.figure(figsize=(12, 12))

for images, labels in train_dataset.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

plt.tight_layout()
plt.show()
```

[3] ✓ 1.9s

... Classes in the Dataset:
['Bacterial_spot', 'Early_blight', 'Healthy', 'Late_blight', 'Leaf_Mold', 'Mosaic_virus', 'Septoria_leaf_spot',



```

model = Sequential([

    keras.Input(shape=(224, 224, 3)),

    # Normalize pixel values
    layers.Rescaling(1./255),

    # First Convolution Block
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    # Second Convolution Block
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    # Third Convolution Block
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    # Flatten
    layers.Flatten(),

    # Dense Layer
    layers.Dense(128, activation='relu'),

    # Dropout
    layers.Dropout(0.5),

    # Output Layer
    layers.Dense(len(class_names), activation='softmax')

])

```

[4] ✓ 0.1s

```

# Compile the CNN model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

[5] ✓ 0.0s

Py

... WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not

```

# Train the CNN model
history = model.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=4
)

```

[6] ✓ 16m 28.1s

Py

... Epoch 1/4
 C:\Users\Swaraj Shedge\anaconda3\Lib\site-packages\keras\src\trainers\epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored
 self.data_adapter = data_adapters.get_data_adapter(
 454/454 ————— 240s 524ms/step - accuracy: 0.5920 - loss: 1.1953 - val_accuracy: 0.7985 - val_loss: 0.5878
 Epoch 2/4
 454/454 ————— 248s 545ms/step - accuracy: 0.7541 - loss: 0.7081 - val_accuracy: 0.8508 - val_loss: 0.4302
 Epoch 3/4
 454/454 ————— 247s 545ms/step - accuracy: 0.8170 - loss: 0.5276 - val_accuracy: 0.8513 - val_loss: 0.4229
 Epoch 4/4
 454/454 ————— 253s 557ms/step - accuracy: 0.8440 - loss: 0.4480 - val_accuracy: 0.8808 - val_loss: 0.3478

```

loss, accuracy = model.evaluate(validation_dataset)

# Display the results
print(f"\nValidation Loss      : {loss:.4f}")
print(f"Validation Accuracy : {accuracy * 100:.2f}%")

```

7] ✓ 18.5s

114/114 ————— 19s 162ms/step - accuracy: 0.8808 - loss: 0.3478

Validation Loss : 0.3478
 Validation Accuracy : 88.08%

```

train_accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

train_loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(1, len(train_accuracy) + 1)

# Accuracy Graph
plt.figure(figsize=(8,5))

plt.plot(epochs_range, train_accuracy, label='Training Accuracy')
plt.plot(epochs_range, val_accuracy, label='Validation Accuracy')

plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.show()

# Loss Graph
plt.figure(figsize=(8,5))

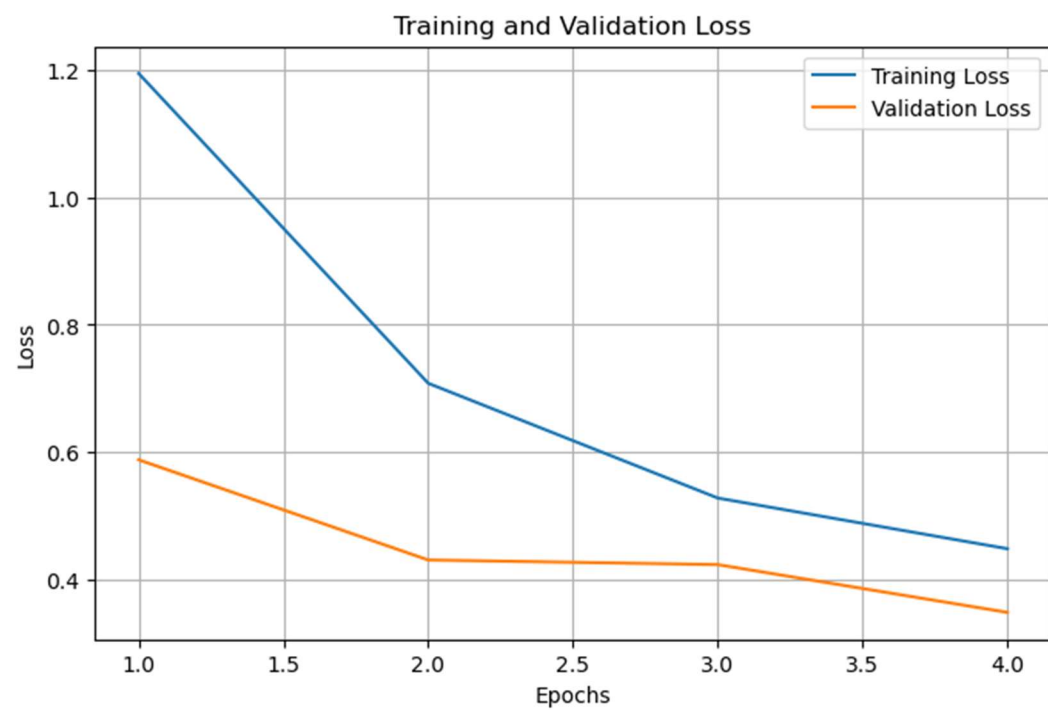
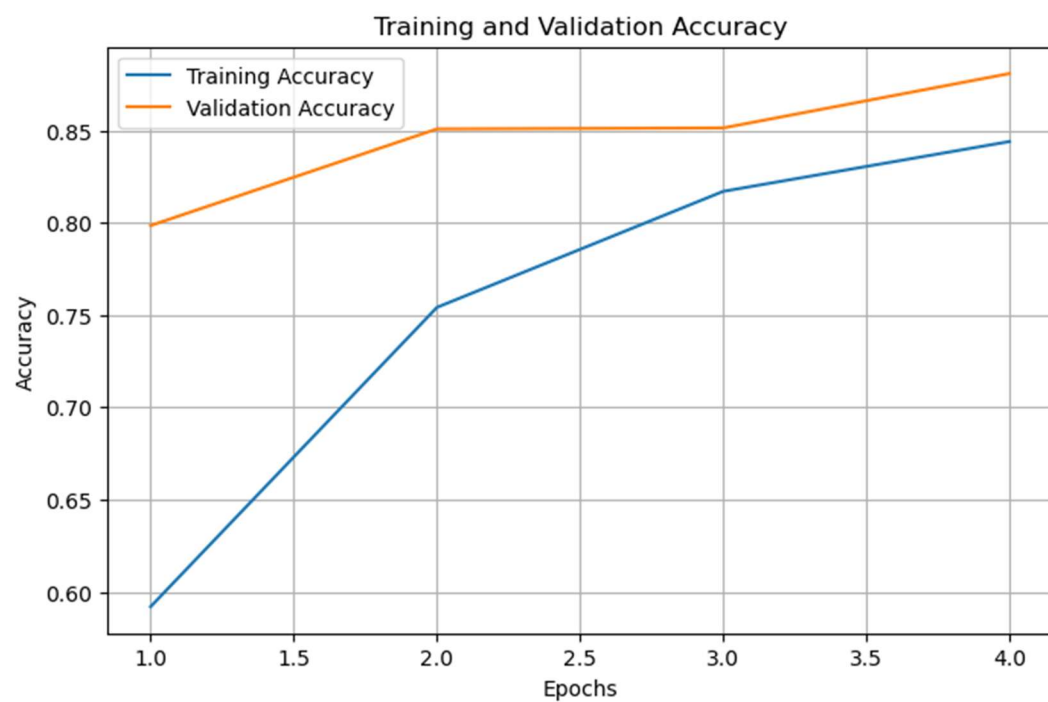
plt.plot(epochs_range, train_loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')

plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.show()

```

[8] ✓ 0.4s



```
from tensorflow.keras.utils import load_img, img_to_array

# Path of the image to predict
image_path = "image.png"
# image_path = "tomato/Late_blight/image (3).jpg"
# Load and resize the image
img = load_img(image_path, target_size=(224, 224))

# Convert image to array
img_array = img_to_array(img)

# Add batch dimension
img_array = np.expand_dims(img_array, axis=0)

# Make prediction
predictions = model.predict(img_array)

# Get predicted class index
predicted_index = np.argmax(predictions[0])

# Get class name
predicted_class = class_names[predicted_index]

# Get confidence score
confidence = np.max(predictions[0]) * 100

# Display image and prediction
plt.imshow(img)
plt.title(f"Predicted: {predicted_class}\nConfidence: {confidence:.2f}%")
plt.axis("off")
plt.show()

print("Predicted Disease :", predicted_class)
print(f"Confidence Score : {confidence:.2f}%")
```

[11] ✓ 0.3s

Predicted: Bacterial_spot
Confidence: 99.63%



Analysis:

- **Task Formulation:** Supervised multi-class image classification for plant pathology.
- **Dataset Volume:** 18,160 images in total across all classes.

- **Data Partitioning:**
 - **Training Set:** 14,528 images (80%)
 - **Validation Set:** 3,632 images (20%)
- **Target Classes (10 Total):**
 - *Healthy*
 - *Bacterial Spot*
 - *Early Blight*
 - *Late Blight*
 - *Leaf Mold*
 - *Mosaic Virus*
 - *Septoria Leaf Spot*
 - *Spider Mites (Two-spotted spider mite)*
 - *Target Spot*
 - *Yellow Leaf Curl Virus*
- **Input Specifications:** Standardized to $224 \times 224 \times 3$ (RGB), making the input compatible with standard transfer learning backbones (e.g., MobileNetV2, ResNet50, EfficientNet).